VPPaaS Sprint 1 Report

João Russo 1116543
António Palma 1119203
Bahareh Bodaghi 1117303

May 2026

1 Introduction

This report presents the first sprint implementation of the VPPaaS platform. The project follows a microservices architecture using Quarkus, Kafka, AWS infrastructure, and AI-supported forecasting services.

2 Information Flows

In this section, we present the information flows for the main business processes of the VPPaaS platform. The sequence diagrams show how Camunda coordinates the process, how Kong forwards requests to the correct microservices, and where Kafka is used for event-driven communication. In the diagrams, the notes indicating "Process request" represent internal processing inside a microservice, usually involving database access.

Since Camunda, Kong and Konga are to be used on Sprint 2 of the project, and since their capabilities are still unknown, the microservices we built migh have to be modified to better fit our use case (whether it is adding, altering endpoints or modifying the database tables).

To build each sequence flow diagram, we used UML.

2.1 Prosumer Management Business Process

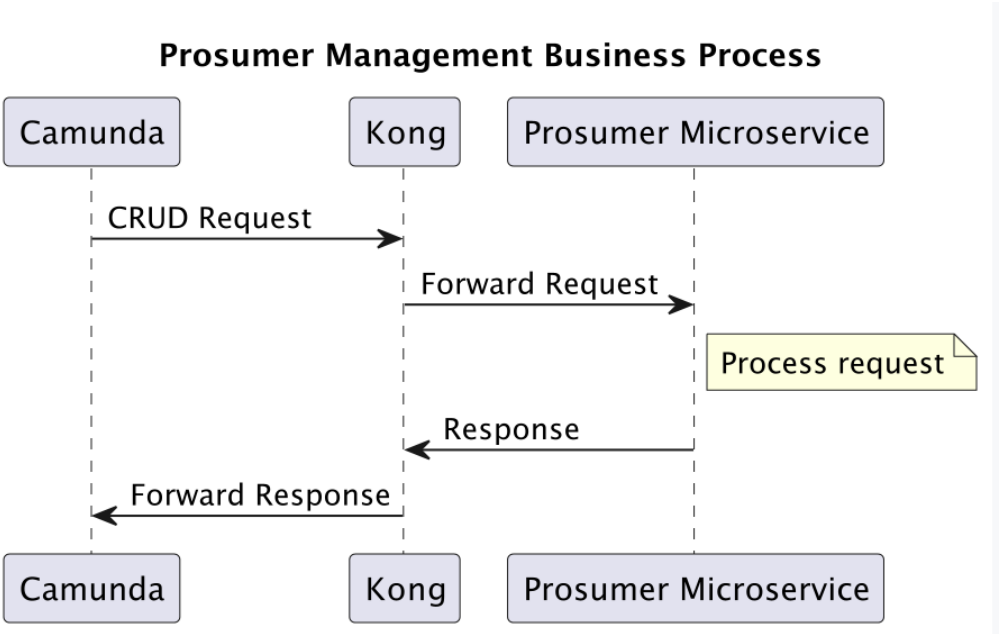

Figure 1: Prosumer Management Business Process

The business process in this case receives the data from the client and either creates a new Prosumer, or receives the Prosumer identifier (and optionally some additional data) so that it can update, delete or read it.

2.2 Utility Operator Management Business Process

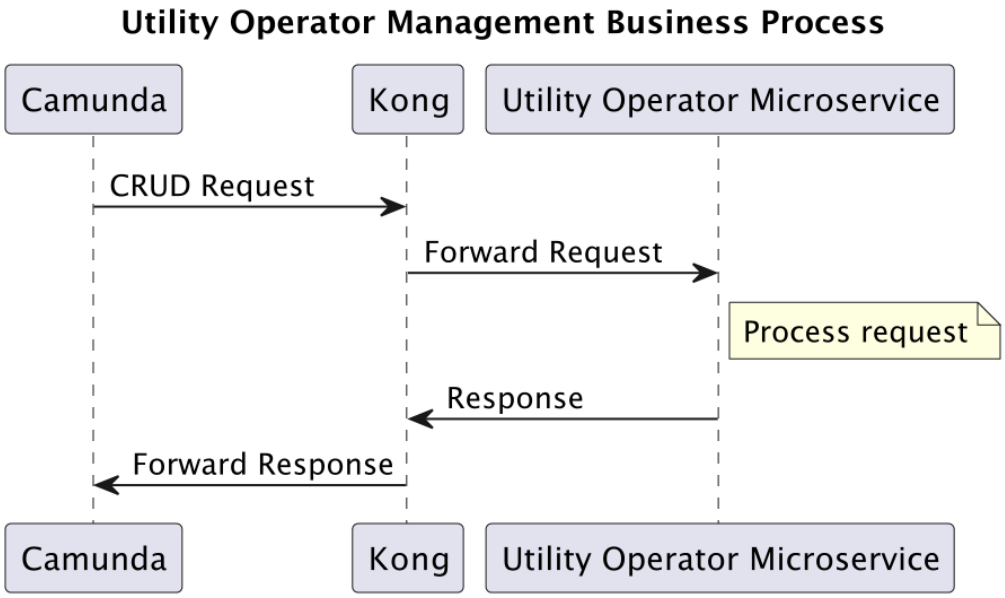

Figure 2: Utility Operator Management Business Process

The business process in this case receives the data from the client and either creates a new Utility Operator, or receives the Utility Operator identifier (and optionally some additional data) so that it can update, delete or read it.

2.3 Grid Cell Management Business Process

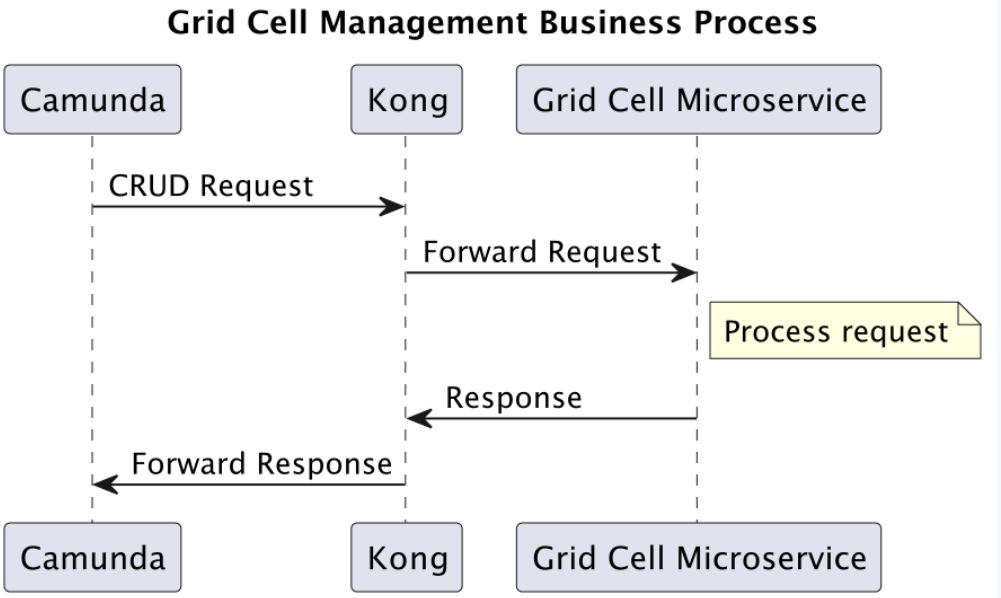

Figure 3: Grid Cell Management Business Process

The business process in this case receives the data from the client and either creates a new Grid Cell, or receives the Grid Cell identifier (and optionally some additional data) so that it can update, delete or read it.

2.4 Asset Management Business Process

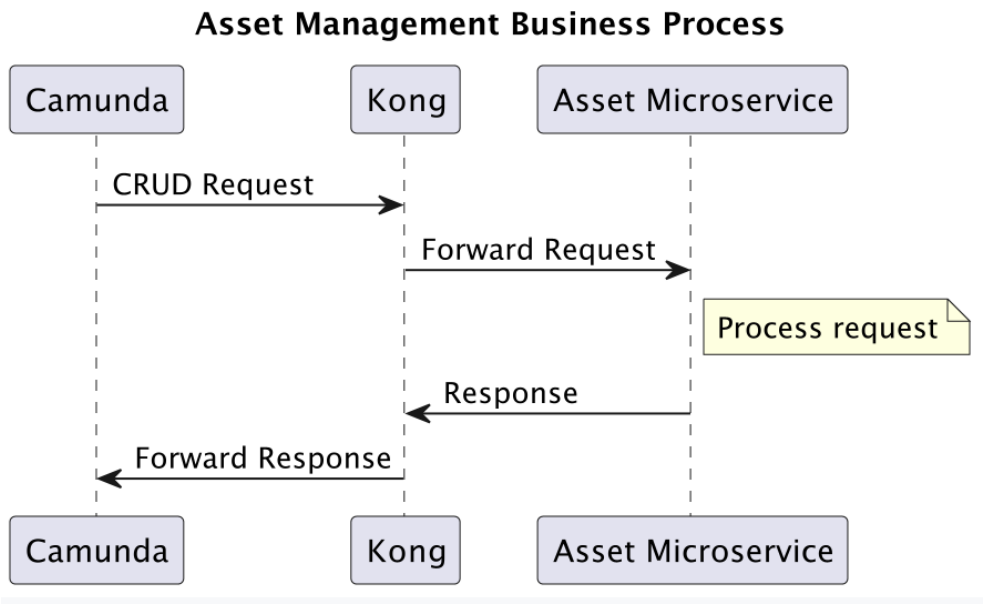

Figure 4: Asset Management Business Process

The business process in this case receives the data from the client and either creates a new Asset, or receives the Asset identifier (and optionally some additional data) so that it can update, delete or read it.

2.5 Asset Link Creation and Reading Business Process

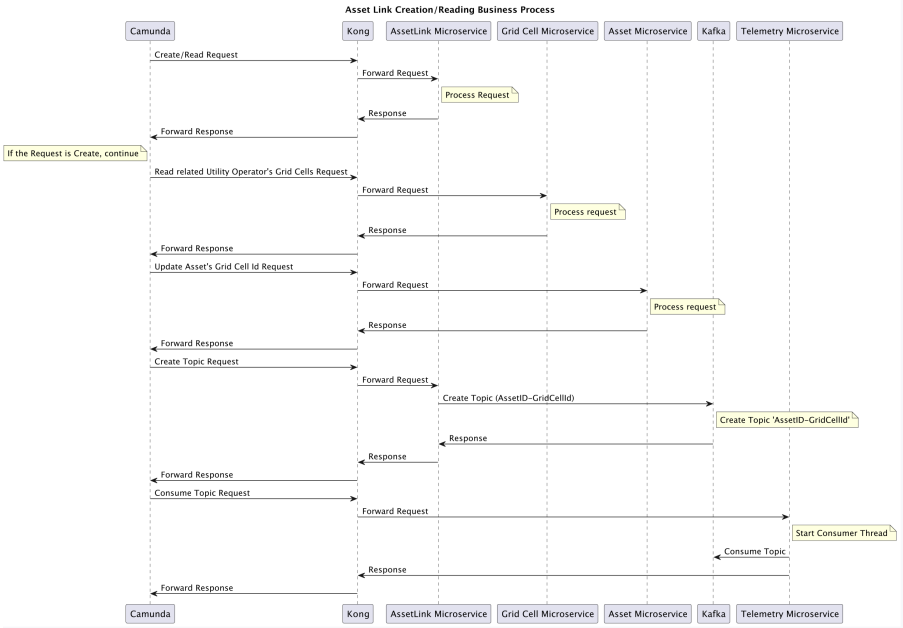

Figure 5: Asset Link Creation and Reading Business Process

This business process manages the creation and reading of asset links, which are contracts between prosumers and utility operators. The flow creates or reads the asset link, updates the related asset information, creates the required topic, and equests the consumption of the new topic's events.

2.6 Asset Link Deletion Business Process

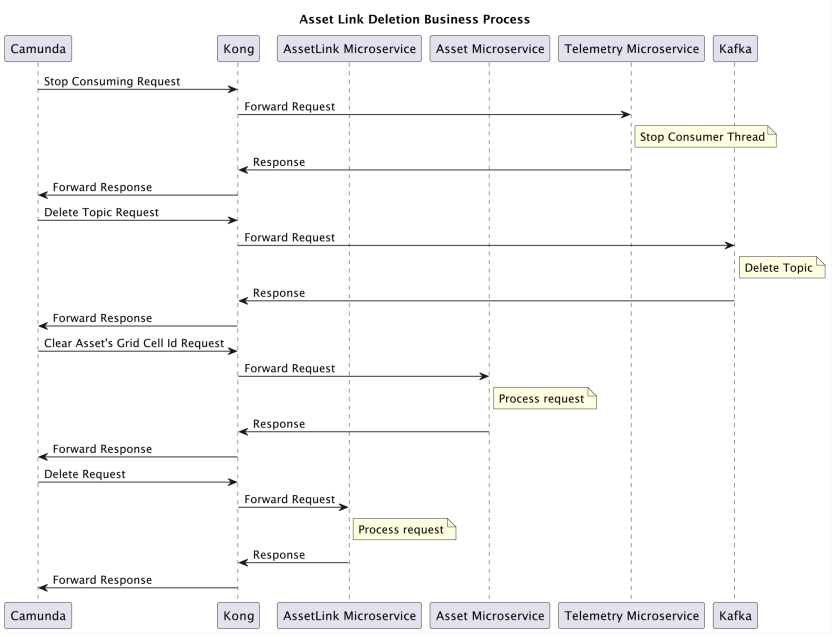

Figure 6: Asset Link Deletion Business Process

This business process manages the deletion of an asset link. When an asset link is removed, the system stops the related telemetry consumption, along with topic deletion, and removes the affected prosumer's asset production target (the grid cell), since its prosumer's contarct with the utility operator is now gone.

2.7 Telemetry Ingestion Business Process

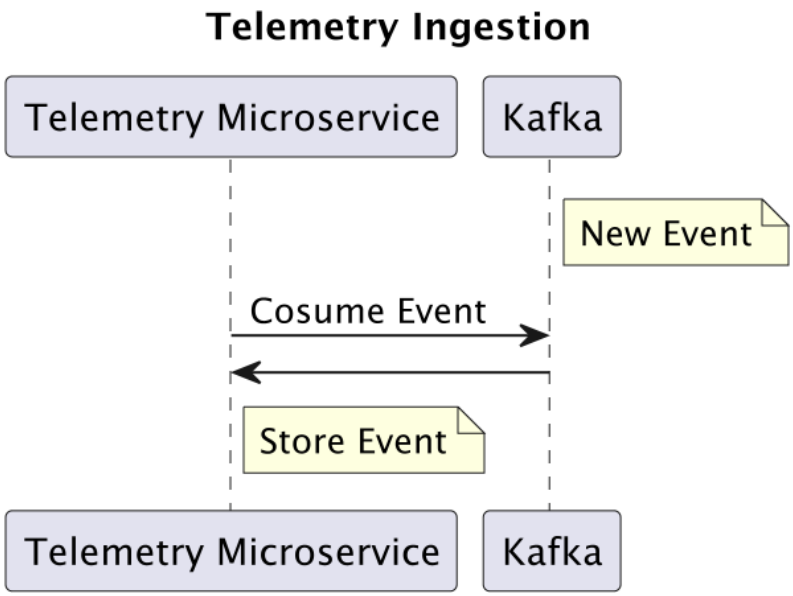

Figure 7: Telemetry Ingestion Business Process

This business process handles telemetry ingestion. Telemetry data is produced by an asset to a Kafka topic and eventually consumed by the Telemetry microservice.

2.8 Flexibility Emission Business Process

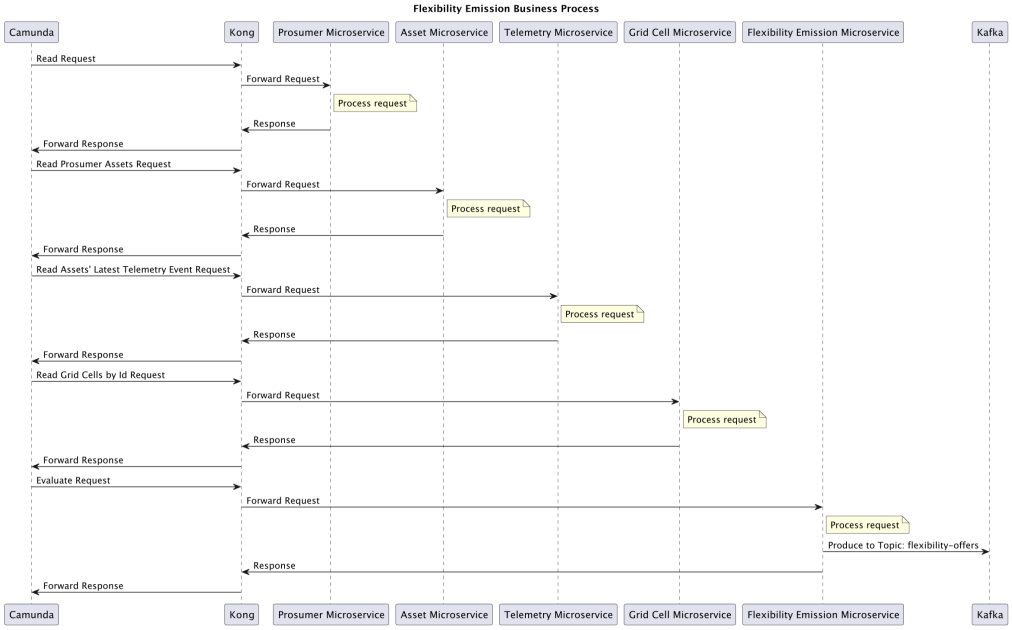

Figure 8: Flexibility Emission Business Process

This business process analyzes telemetry and asset data to decide whether a flexibility event (SELL or mark as UNAVAILABLE) should be emitted. First we collect prosumer, its assets, their latest telemetry events and the grid cell data the assets are currently targeting. When the required conditions are met, new event is sent to the flexibility-offers Kafka topic.

2.9 Grid Balancing Recommendation Business Process

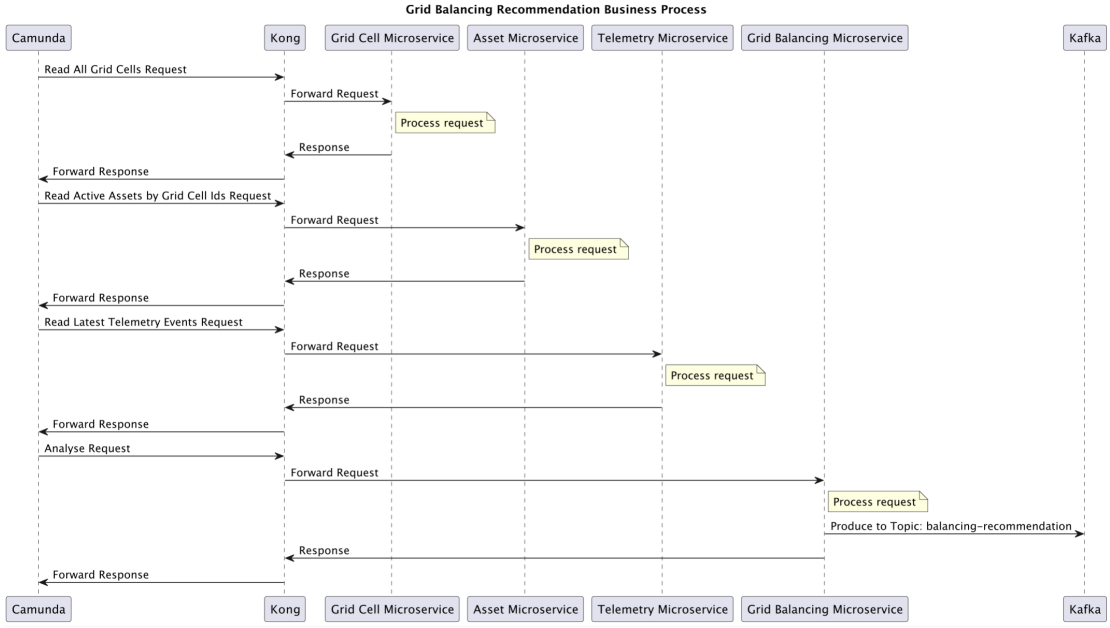

Figure 9: Grid Balancing Recommendation Business Process

This business process analyzes the state of different grid cells and produces recommendations to balance the grid. For that we need the grid cell data, assets currently targeting those cells and their latest telemetry events to those cells. If one grid cell is under stress and another has available capacity, the system can recommend shifting load between them, done through a Kafka topic event.

2.10 Energy Analytics Business Process

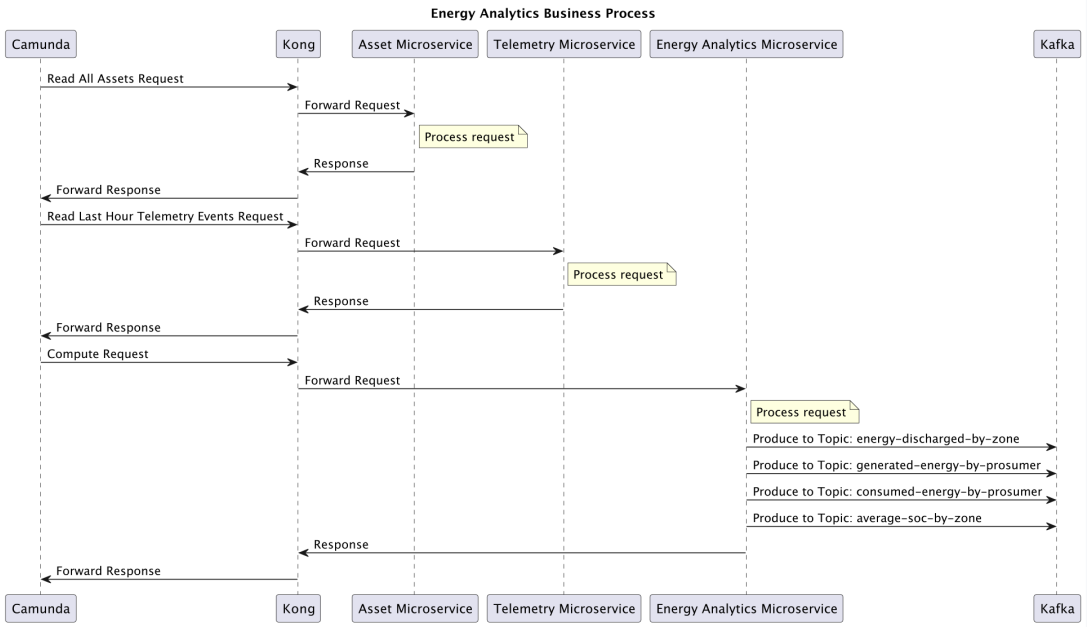

Figure 10: Energy Analytics Business Process

This business process produces energy analytics based on the data collected by the platform. It aggregates information such as energy generated by prosumer, energy consumed by prosumer, energy discharged by zone and average state of charge, supporting later analysis and decision-making in the VPPaaS platform. To this end, we collect asset data and their telemetry events in the last hour, and compute metrics on it.

2.11 Flexibility Forecasting Business Process

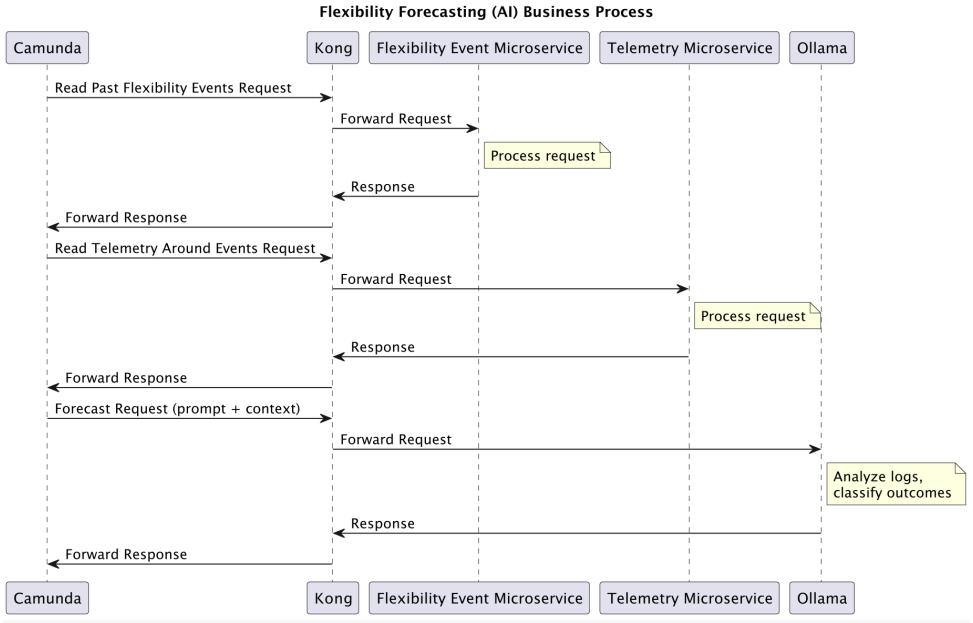

Figure 11: Flexibility Forecasting Business Process

This business process use of LLMs to analyze past flexibility event and determine if their success rate. The process first collects flexibility events and telemetry "around" those event's timestamps (10 events before and 10 after). This data is then sent and analysed by a model running in Ollama.

3 System Implementation

In this section, we walk you through the various implementation decisions taken for all parts of our project.

3.1 Kafka

The Kafka cluster is deployed through Terraform, located in the terraform/kafka directory. The cluster is composed of 3 brokers with a default replication factor of 3. This ensures high availability and fault tolerance, allowing the cluster to survive broker failures without losing data.

By default, topics in our cluster are divided into 3 partitions. This design choice enables read parallelism; if event throughput increases in the future, we can scale up our applications to have up to 3 consumers within a single consumer group processing a topic's messages concurrently.

Our Kafka configuration script (terraform/kafka/creation.sh) handles the cluster setup and applies these default values. It configures the number of partitions per topic, the replication factor, the cluster quorum configuration, the broker IDs, and the broker file system.

If we ever need to scale the cluster by adding more brokers, we will maintain the replication factor at 3 rather than matching the new broker count. This ensures we maintain strong fault tolerance (tolerating up to 2 broker failures) while preventing excessive storage overhead.

3.2 RDS

Amazon RDS is used as the managed relational database service. The microservices use MySQL databases hosted on AWS RDS for persistent storage.

Database access is configured through Quarkus datasource properties and, during deployment, the database URL is passed to the containers as an environment variable.

3.3 Ollama

We are using Ollama and a Llama model for better flexibility forecasting support. This service lives on an EC2 instance

The deploy script exposes Ollama through port `11434`, using the `/api/generate` endpoint.

3.4 Microservices

The platform follows a Quarkus-based microservices architecture, where each business domain is isolated into its own independent service to allow for independent scalability and separation of responsibilities. Services are packaged using Maven, containerized as Docker images, pushed to Docker Hub, and deployed on AWS EC2 instances.

3.4.1 Overview

The current microservices endpoints and their intended usage are:

- **Asset** — manages physical assets (batteries, solar panels, EV chargers) associated with prosumers and possibly an operator's grid cell.
- **AssetLink** — creates contarcts between Prosumers and Operators, allowing prosumer assets to produce to utility operator's grid cells. It also coordinates Kafka topic creation for new contracts.
- **EnergyAnalytics** — aggregates telemetry data to compute energy metrics such as discharge, generation, consumption, and average state of charge.
- **FlexibilityEmission** — evaluates battery telemetry and emits flexibility events signalling if power should be sold or not.
- **GridBalancingRecommendation** — analyses grid cell conditions and produces energy transfer recommendations between cells.
- **GridCell** — manages electrical grid cell information used to organize assets geographically.
- **Prosumer** — manages prosumers, representing entities capable of both producing and consuming energy.
- **Telemetry** — dynamically consumes telemetry data from Kafka topics and stores them as records for downstream processing.
- **UtilityOperator** — manages utility operator information, associated with grid operation activities.

3.4.2 Implementation

The following table summarises the implemented microservice endpoints and their responsibilities.

Microservice	Method	Endpoint	Description
`Asset`	GET	`/Asset`	Get all assets

Continued on next page

Continued from previous page

Microservice	Method	Endpoint	Description
	GET	/Asset/{id}	Get asset by ID
	POST	/Asset	Create asset
	POST	/Asset/{id}	Update asset
	DELETE	/Asset/{id}	Delete asset
	GET	/Asset/active/by-grid-cell-ids	Get active assets by grid cell IDs
	GET	/Asset/active/by-prosumer/{id}	Get active batteries by prosumer
AssetLink	GET	/AssetLink	Get all asset links
	GET	/AssetLink/{id}	Get asset link by ID
	GET	/AssetLink/{idProsumer}/{idUtilityOperator}	Get asset link by prosumer and operator
	POST	/AssetLink	Create asset link
	DELETE	/AssetLink/{id}	Delete asset link
	POST	/AssetLink/topic/{assetId}/{gridCellId}	Create Kafka topic for asset link
	DELETE	/AssetLink/topic/{assetId}/{gridCellId}	Delete Kafka topic for asset link
Energy Analytics	GET	/EnergyAnalytics	Get all analytics records
	POST	/EnergyAnalytics/analyse	Run energy analysis and store results
Flexibility Emission	GET	/FlexibilityEmission	Get all flexibility events
	POST	/FlexibilityEmission/evaluate	Evaluate and emit flexibility events
GridBalancing Recommendation	GET	/GridBalancingRecommendation	Get all balancing recommendations
	POST	/GridBalancingRecommendation/balance	Calculate and emit balancing recommendations
GridCell	GET	/GridCell	Get all grid cells
	GET	/GridCell/{id}	Get grid cell by ID
	POST	/GridCell	Create grid cell
	PUT	/GridCell/{id}	Update grid cell
	DELETE	/GridCell/{id}	Delete grid cell
	GET	/GridCell/by-ids	Get grid cells by IDs
	GET	/GridCell/by-operator-ids	Get grid cells by operator IDs
Prosumer	GET	/Prosumer	Get all prosumers
	GET	/Prosumer/{id}	Get prosumer by ID
	POST	/Prosumer	Create prosumer
	PUT	/Prosumer/{id}/{name}/{FiscalNumber}/{location}	Update prosumer
	DELETE	/Prosumer/{id}	Delete prosumer
Telemetry	GET	/Telemetry	Get all telemetry events
	GET	/Telemetry/last-hour	Get telemetry from the last hour
	GET	/Telemetry/asset/{assetId}/around/{timestamp}	Get telemetry around a timestamp for a specific asset
	POST	/Telemetry/latest	Get latest telemetry for asset/grid cell pairs
	POST	/Telemetry/consume	Start Kafka consumer for a topic
	POST	/Telemetry/stop	Stop Kafka consumer for a topic
Utility Operator	GET	/UtilityOperator	Get all utility operators
	GET	/UtilityOperator/{id}	Get utility operator by ID
	POST	/UtilityOperator	Create utility operator
	PUT	/UtilityOperator/{id}	Update utility operator
	DELETE	/UtilityOperator/{id}	Delete utility operator

3.4.3 Design Decisions

Kafka Topic Auto-Creation Automatic Kafka topic creation is disabled as if it were a production environment. Topics must instead be provisioned explicitly via the dedicated endpoint. This ensures to dangling topics are ever created, wasting resources.

No Update Operation on AssetLink An update operation for AssetLink was intentionally omitted. Because an AssetLink represents a binding contract between a Prosumer and a Utility Operator, it is treated as immutable. The contract is either actively valid or it must be deleted. Therefore, ongoing contracts are never updated; they can only be created or removed.

Foreign Key Constraints Not Enforced Each microservice strictly owns its own data and must not depend on another microservice's data. Enforcing physical foreign key constraints across service boundaries creates tight coupling that severely hinders scalability. If a specific service experiences massive growth and requires its database to be migrated to a dedicated, higher-performance server, cross-database foreign keys would make this physical separation impossible.

Failed Telemetry Consumer Service Startup Logic was added to the Telemetry service that ensures that if the microservice (or a replica of it) ever goes down, and making it possible to resume topic consumption (starting at the last offset commited by the consumer). This was achieved by storing tracking service-to-topic consumption data with a new `TopicSubscription` table, a new Quarkus `recovery.failed-service-uuids` variable (which allows a list of comma-separated UUIDs to be passed, these UUIDs identify the previously failed microservice) and Telemetry microservice logic on startup, which makes the current service take ownership of all topic consumption from the failed previously faile microservice. Ensuring continuous data processing, preventing message loss, and maintaining high availability within the telemetry pipeline.

4 Tests

The project includes automated tests implemented inside the Quarkus microservices using JUnit, `@QuarkusTest`, RestAssured, Mockito and Test Containers (for the MySQL database and Kafka). The tests are located under each microservice's `src/test/java` directory.

Current unit test suites are:

- `AssetResourceTest.java`
- `AssetLinkResourceTest.java`
- `KafkaTopicServiceTest.java`
- `EnergyAnalyticsResourceTest.java`
- `FlexibilityEmissionResourceTest.java`
- `GridBalancingRecommendationResourceTest.java`
- `GridCellResourceTest.java`
- `ProsumerResourceTest.java`
- `TelemetryResourceTest.java`
- `DynamicTopicConsumerTest.java`
- `KafkaConsumerServiceTest.java`
- `TopicSubscriptionRecoveryServiceTest.java`
- `UtilityOperatorResourceTest.java`

The tests validate REST endpoints, database interactions, and Kafka-related operations. The Asset microservice tests illustrate the general testing approach used throughout the platform.

Before each Asset test, the database table is cleaned and populated with sample records. The tests then validate endpoints such as:

- `GET /Asset`
- `GET /Asset/{id}`
- `POST /Asset`
- `POST /Asset/{id}`
- `DELETE /Asset/{id}`

The tests verify both successful and error scenarios, including validation of HTTP status codes and response contents.

We have also added integration tests, which you can run after the whole environment is up. These were designed to test telemetry consumption via Kafka and event microservice event production. The provided event producer was also changed and allows more fields to be passed in order to have a more accurate event production.

To run all of these, unit and integration tests, run:

```
./test.sh
```

5 Project Structure and Deployment

The project is organised into four main areas: business-process diagrams, Quarkus microservices, Terraform infrastructure code, and deployment support scripts. This separation makes the repository easier to navigate and helps distinguish application logic from infrastructure and operational tooling.

5.1 Top-Level Structure

```
project/
|-- diagrams/                       # PlantUML sequence diagrams
|-- microservices/                  # Quarkus microservices
|   |-- Asset/
|   |-- AssetLink/
|   |-- EnergyAnalytics/
|   |-- FlexibilityEmission/
|   |-- GridBalancingRecommendation/
|   |-- GridCell/
|   |-- Prosumer/
|   |-- Telemetry/
|   `-- UtilityOperator/
|-- integration-tests/
|-- terraform/
|   |-- kafka/                      # Kafka cluster deployment
|   |-- rds/                        # RDS database deployment
|   `-- microservices/              # EC2 deployment per microservice
|-- logs/                           # Terraform deployment logs
|-- deploy.sh                       # Full deployment script
|-- undeploy.sh                     # Full teardown script
|-- access.sh                       # Docker credentials setup
`-- addresses.sh                    # Retrieve deployed service addresses
```

The `diagrams/` directory contains the PlantUML sources used to document the platform's information flows. The `microservices/` directory groups the Quarkus services by business domain. The `integratio-tests/` directory contains integration tests, using bash scripts. The `terraform/` directory contains the infrastructure-as-code modules used to provision Kafka, RDS, and the EC2 instances that host the services. Finally, the shell scripts at the project root support deployment, teardown, authentication, and discovery of deployed service addresses.

5.2 Microservice Internal Structure

Each microservice follows a shared internal layout based on Quarkus and Maven conventions, which keeps the services consistent and easier to maintain.

```
<ServiceName>/
|-- pom.xml                         # Maven build configuration
|-- src/
|   |-- main/
|   |   |-- java/org/acm(e)/        # REST resources and business logic
|   |   `-- resources/
|   |       `-- application.properties
|   `-- test/
|       `-- java/org/acm(e)/        # JUnit tests
`-- README.md
```

The `pom.xml` file defines dependencies and build configuration. The `src/main/java` directory contains the REST resources, service classes, and domain logic, while `src/main/resources/application.properties` stores runtime configuration. The `src/test/java` directory contains the automated test suites, and the `README.md` file documents service-specific usage and implementation details.

Deployment and configuration are managed through Terraform modules and shell scripts. Infrastructure parameters such as AWS resources, Kafka broker count, database connection settings, and service endpoints are configured through Terraform variables and Quarkus `application.properties` files.

5.3 Deployment Procedure

The platform deployment process is performed through shell scripts and Terraform modules.

1. Configure AWS credentials in:

   ```
   terraform/.aws/.credentials
   ```

2. Copy both aws PEM key to terraform/.aws/.

3. Create `access.sh` from `access.sh.example` and define the Docker Hub credentials:

   ```
   export DockerUsername=
   export DockerPassword=
   ```

4. Execute the authentication script:

   ```
   source ./access.sh
   ```

5. Deploy the infrastructure and microservices:

   ```
   ./deploy.sh
   ```

6. Retrieve the deployed service addresses:

   ```
   ./addresses.sh
   ```

7. Remove the deployed infrastructure:

   ```
   ./undeploy.sh
   ```